

Theory Questions & Answers - Oly

NLP - NATURAL LANGUAGE PROCESSING (Q1-Q3)

Q1. What is tokenization? Give an example - show how the sentence "I'm learning NLP in 2025!" would be tokenized. Expected: a list of tokens and a one-sentence explanation of what tokenization does and why it's the first step in NLP.

Answer:

Tokenization breaks text into smaller pieces such as words, punctuation, or subwords. For example, "I'm learning NLP in 2025!" can be split as ["I", "'m", "learning", "NLP", "in", "2025", "!"]. It comes first because a model needs tokens to work with text instead of raw characters.

Q2. What is the difference between stemming and lemmatization? Apply both to the words "running" and "better" and explain which preserves more linguistic meaning. Hint: stemming is fast but crude (cuts off suffixes). Lemmatization uses a dictionary to find the base form.

Answer:

Stemming is a rough shortcut. It chops off endings, so "running" often becomes "run", and "better" may stay as "better". Lemmatization is smarter. It uses word forms and a dictionary, so "running" becomes "run" and "better" becomes "good". That makes lemmatization better at preserving real meaning.

Q3. What does TF-IDF stand for? Explain in plain language why the word "the" scores almost zero in TF-IDF, while the word "photosynthesis" would score high. Think about: what makes a word useful for identifying a specific document among many documents?

Answer:

TF-IDF stands for Term Frequency × Inverse Document Frequency. It gives a word more weight when it appears a lot in one document but not many others. "The" shows up everywhere, so it gets a low score. "Photosynthesis" is rarer and more specific, so it gets a much higher score and helps identify the right document.

VECTOR DATABASES & EMBEDDINGS (Q4-Q6)

Q4. What is a sentence embedding? How is it fundamentally different from one-hot encoding? Give one advantage embeddings have that one-hot vectors don't. Think about: what can you measure between two embeddings that you can't measure between two one-hot vectors?

Answer:

A sentence embedding is a numeric vector that captures meaning. One-hot encoding is much more basic, because it only marks whether something is present or not. Embeddings are useful because they let you compare meanings (via cosine similarity, distance, etc.), not just check exact matches.

Q5. Explain cosine similarity in plain language. If two document vectors point in almost the same direction, what does that tell us about the documents they represent? You don't need to write the formula - a geometric or intuitive explanation is enough. Bonus: When might Euclidean distance be a poor choice for comparing embeddings?

Answer:

Cosine similarity looks at the direction of two vectors (the angle between them). If they point in almost the same direction, the documents are probably about similar ideas or topics. *Bonus:* Euclidean distance can be a poor choice when the vectors differ a lot in length (magnitude), because the size difference can dominate the comparison even when the underlying meaning is similar.

Q6. Why can't a regular SQL query like `WHERE description LIKE '%pizza%'` find semantically similar documents? What does a vector index solve that SQL can't? *Example: "Italian food" and "pasta and risotto" - one contains the word, the other doesn't, but they're about the same topic.*

Answer:

A normal SQL search only matches exact words or patterns you give it. It cannot understand that two different phrases may be talking about the same idea. A vector index solves this by comparing embeddings (semantic vectors) and finding content with similar meaning, even when the exact wording is completely different.

RAG - RETRIEVAL-AUGMENTED GENERATION (Q7-Q8)

Q7. What problem does RAG solve that a plain LLM (without RAG) cannot? Give a concrete example of when you would choose RAG over just prompting the LLM directly. Think about: knowledge cutoffs, private data, hallucinations, up-to-date information.

Answer:

RAG helps when the model needs real, current, or private information that it doesn't have in its training data. It gives the model actual documents to read at query time. A good example is asking about an internal company policy or the latest product specs - RAG can pull the exact relevant passage instead of the LLM making something up (hallucinating) or saying it doesn't know because of a knowledge cutoff.

Q8. Describe the 3 main steps of a RAG pipeline in the correct order. Be clear about what happens at ingestion time (when you load documents) vs query time (when a user asks a question). The 3 steps are: chunk → embed → store (ingestion) and embed query → retrieve → generate (query time).

Answer:

Ingestion time: You split the documents into smaller chunks, turn those chunks into embeddings (vectors), and store them in a vector database.

Query time: You embed the user's question into a vector, retrieve the most similar chunks from the vector database, and pass those chunks to the LLM so it can generate an answer grounded in the retrieved context.

DOCKER + AI AGENTS (Q9-Q10)

Q9. What is the difference between a Docker image and a Docker container? Use an analogy to explain.

Common analogies: class vs instance, recipe vs cake, blueprint vs building. Any analogy that works is fine.

Answer:

A Docker image is like a recipe or a blueprint - it contains all the instructions and ingredients needed to create something, but it isn't running yet. A Docker container is the actual running instance you get when you "bake" that recipe. It's the live, working application with its own isolated filesystem, network, and processes.

Q10. What is the difference between a simple LLM chatbot and an AI agent with tools? Give one concrete example of a "tool" and explain why it makes the agent more capable. Think about: what can an agent do that a plain LLM cannot? (e.g., look things up in real-time, run code, query a database, send an email...)

Answer:

A simple chatbot mostly just generates text based on what it was trained on. An AI agent can also use tools - external functions it can call to interact with the real world. Example tool: a web search tool or a calculator. This makes the agent far more capable because it can fetch live information, perform calculations, query databases, or take actions (like sending an email or updating a spreadsheet) instead of only talking about them.

MCP & AGENT SKILLS (Q11-Q12)

Q11. What is MCP (Model Context Protocol)? What problem does it solve for AI coding assistants like GitHub Copilot? Name two examples of things an MCP server might expose to an AI assistant. Think about: how does a language model normally know about your database, your files, or your GitHub issues? What does MCP standardize?

Answer:

MCP (Model Context Protocol) is a standard that lets AI coding assistants securely connect to external tools and data sources. It solves the problem that language models normally have no built-in way to access your local files, databases, GitHub issues, or other context in a consistent, secure manner. Two examples of what an MCP server might expose: (1) a file system server (read/write files in your project), and (2) a database query server (run SQL or vector searches).

Q12. What are Agent Skills in the context of AI coding assistants? How are they different from just writing instructions in a plain prompt? Show a minimal example of what a skill's .md metadata block might look like. Think about: a skill has a name, a description, and a path to a detailed instructions file - it lets the AI decide when to activate domain-specific knowledge automatically.

Answer:

Agent Skills are reusable, structured pieces of guidance that help an AI assistant know when and how to use a specific capability or domain knowledge. Unlike a plain prompt (which is just text the model sees every time), a skill is registered with metadata so the assistant can automatically decide to activate it based on the user's request.

Example YAML-style metadata block:

```
name: pet-store-assistantdescription: Help with pet store inventory, orders, and customer support questions.file: skills/pet-store-assistant/SKILL.md
```

Example XML-style skill definition:

```
<skill> <name>pet-store-assistant</name> <description>Help with pet store inventory, orders, and customer support questions. Use when the user asks "How do I check stock?" or "What is the status of this order?"</description> <file>path/to/skills/pet-store-assistant/SKILL.md</file></skill>
```
